

Object, Class & Encapsulation

A. Class & Object

1. What exactly happens internally when you create an object using `obj = Employee()`? Explain the complete flow.
2. In Python, everything is an object. Then what is a class itself? Is a class also an object? Explain.
3. What is the difference between `Employee()` and `obj = Employee()` from a memory/object perspective?
4. Suppose two objects are created from the same class. How does Python ensure that changing `e1.name` doesn't normally change `e2.name`?

```
e1 = Employee()  
e2 = Employee()
```

5. What is the actual purpose of `self`? Why can't Python automatically identify the current object without passing `self`?
6. What are the values of `a.x` and `b.x`? Why?

```
class Test:  
    x = 10  
  
a = Test()  
b = Test()  
  
a.x = 20
```

7. What happens if you access an attribute through an object when that attribute doesn't exist in the object's instance dictionary?
8. Can you create an object without explicitly defining `__init__()`? If yes, what happens when the object is created?
9. What is the difference between an instance attribute and a class attribute, and when can using a class attribute accidentally create a bug?
10. What will `s2.subjects` contain and why?

```
class Student:  
    subjects = []  
  
s1 = Student()  
s2 = Student()  
  
s1.subjects.append("Python")
```

11. What is the difference between `__new__` and `__init__`? Which one runs first, and which one actually creates the object?
12. What does `type(Employee)` return? What is `type(type)`? Explain the relationship between objects, classes and metaclasses.
13. What is the difference between `==` and `is` when comparing two objects of the same class? How can you make `==` compare attribute values?

```
e1 = Employee("Ravi")
e2 = Employee("Ravi")

print(e1 == e2)
print(e1 is e2)
```

14. What is `__dict__`? How can you use it to inspect or modify an object's attributes at runtime?
15. What is the difference between a bound method and a plain function? What does `Employee.show` return compared to `obj.show`?
16. Explain the difference between an instance method, a `@classmethod` and a `@staticmethod`. When would you use each one?
17. What is the output, and why is this considered a bug?

```
class Cart:
    def __init__(self, items=[]):
        self.items = items

c1 = Cart()
c2 = Cart()
c1.items.append("apple")
print(c2.items)
```

18. What is the difference between `__str__` and `__repr__`? Which one is used when you call `print(obj)`?
19. Can you add new attributes to an object after it is created? What stops you from doing this, if anything?
20. What happens when you use `del obj`? When does Python actually remove the object from memory, and when is `__del__` called?

B. Encapsulation

21. Python doesn't have truly private variables like some languages. Then how does Python implement encapsulation?
22. Explain the difference between `self._salary` and `self.__salary`. Is either one genuinely private?

23. What is name mangling in Python? Why does Python use it for double-underscore attributes?

24. Can someone outside the class access `__salary`? If yes, how? And if they can, why do we still call it encapsulation?

```
class Employee:
    def __init__(self):
        self.__salary = 50000
```

25. If Python allows access to a "private" variable using name mangling, does that mean Python's encapsulation is weak? Defend your answer as an interviewer would expect.

26. What is the use of `@property`? How do getter and setter work together to control access to an attribute?

```
class Account:
    def __init__(self):
        self._balance = 0

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("Invalid")
        self._balance = value
```

27. Why use a property when you can simply keep the attribute public? How does a property let you change the internal logic later without breaking existing code?

28. What does `__slots__` do? How does it affect `__dict__` and memory usage?

29. What will this code print and why? What does it tell you about name mangling and inheritance?

```
class Parent:
    def __init__(self):
        self.__x = 1

class Child(Parent):
    def __init__(self):
        super().__init__()
        self.__x = 2

c = Child()
print(c.__dict__)
```

30. Design question: you are building a `BankAccount` class. How would you protect the balance from invalid changes? Which encapsulation features of Python would you use and why?

Answers

1. Python first evaluates `Employee`, which is a class object, and calls it. This goes to `type.__call__(Employee)`. It does three things in order: (1) calls `Employee.__new__` (normally `object.__new__`), which allocates memory and creates a new empty instance with its own `__dict__`; (2) calls `__init__(obj, ...)` on that new instance to set up its attributes; (3) returns the finished object, and the name `obj` is bound to it (reference count becomes 1).
2. Yes, a class is also an object. It is created at runtime when the `class` statement runs, it lives in memory, it can be stored in a variable, passed to functions, and have attributes. A class is an instance of its metaclass, which by default is `type`. So `type(Employee)` gives `type`.
3. `Employee()` creates an object but no name points to it, so its reference count drops to 0 right after the expression and CPython frees it immediately. `obj = Employee()` creates the same object and also binds the name `obj` to it, so the object stays alive as long as `obj` refers to it. A variable name is only a reference, not the object itself. You can verify with `id(obj)` and `sys.getrefcount(obj)`.
4. Every instance has its own `__dict__` that stores its attributes. Both objects share the same class (and its methods) but not their data. `e1.name = "A"` writes only into `e1.__dict__`, so `e2.__dict__` is untouched. They are two different objects at two different memory locations (`e1 is e2` is `False`).
5. `self` is a reference to the current instance, passed as the first argument to instance methods. `obj.show()` is internally converted to `Employee.show(obj)`. Methods are plain functions stored once on the class and shared by all objects, so the function has no other way of knowing which object's data to use. Python chooses to pass it explicitly ("explicit is better than implicit").
6. `a.x` is 20 and `b.x` is 10. The line `a.x = 20` does not change the class attribute; it creates a new instance attribute `x` inside `a.__dict__` that shadows the class attribute. `b` has no own `x`, so it still finds `Test.x = 10`.
7. Python looks for the attribute in this order: (1) the instance `__dict__`, (2) the class, (3) the parent classes following the MRO. If it is still not found, Python calls `__getattr__` if the class defines it; otherwise it raises `AttributeError`. (Data descriptors such as properties on the class are checked before the instance dictionary.)
8. Yes. If the class has no `__init__`, Python uses the one inherited from `object`, which does nothing. The object is created normally with an empty `__dict__`. If you pass arguments to such a class, you get a `TypeError`, because `object.__init__` accepts none.
9. An instance attribute belongs to one object, is stored in that object's `__dict__`, and is usually set in `__init__` using `self`. A class attribute is defined in the class body, is stored on the class, and is shared by all instances. A bug happens when the class attribute is mutable (a list or dict): changing it through one object changes it for every object. Another trap is that assigning through an instance (`obj.x = 5`) creates a new instance attribute instead of changing the class attribute.

10. It contains `["Python"]`. `subjects` is a class attribute, so `s1` and `s2` share the same list. `append` changes that list in place (there is no assignment, so nothing is shadowed). Fix: create the list per object inside `__init__`.

```
def __init__(self):
    self.subjects = []
```

11. `__new__` creates and returns the new instance; its first argument is the class. `__init__` only initialises an instance that already exists; its first argument is `self` and it must return `None`. `__new__` runs first, and if it returns an instance of the class, `__init__` is then called on it. `__new__` is rarely overridden, mainly for immutable types (`int`, `str`, tuple subclasses) and singletons.
12. `type(Employee)` returns `<class 'type'>`, and `type(type)` is also `type`. An object is an instance of a class, a class is an instance of a metaclass, and the default metaclass is `type`. `type` is its own metaclass, which is where the chain ends: `obj -> Employee -> type -> type`.
13. `==` calls `__eq__`, which by default compares identity, so it behaves like `is`. `is` checks whether both names point to the same object. Here both lines print `False`, because they are two separate objects. To compare by value, define `__eq__` to compare the attributes (and also `__hash__` if the object is used in sets or as a dict key).
14. `__dict__` is a dictionary that holds the instance attributes. `obj.__dict__` shows something like `{"name": "Ravi"}`. You can read it, or change it: `obj.__dict__["age"] = 25` creates the attribute `age`. `vars(obj)` gives the same result. The class has its own `__dict__` that holds methods and class attributes.
15. `Employee.show` is just a plain function (Python 3 has no unbound methods), so you must pass the instance yourself. `obj.show` is a bound method: the function packed together with the instance (`obj.show.__self__ is obj`), so calling it passes `obj` as `self` automatically. This works because functions implement the descriptor protocol (`__get__`).
16. An instance method takes `self` and works with one object's data. A `@classmethod` takes `cls` and works with the class itself; it is used for alternative constructors like `from_string()` and for class-level data, and it works correctly with inheritance. A `@staticmethod` takes neither; it is just a normal function placed inside the class because it is logically related (helpers, validations).
17. It prints `["apple"]`. Default argument values are evaluated only once, when the function is defined, so every object created without arguments shares the same list. Fix: use `items=None` as the default and create the list inside `__init__`.

```
def __init__(self, items=None):
    self.items = items if items is not None else []
```

18. `__str__` gives a readable string for end users; `__repr__` gives an unambiguous string for developers (ideally one that looks like the code to recreate the object). `print(obj)` and `str(obj)` use `__str__`; if it is not defined, Python falls back to `__repr__`. The interactive shell and debuggers show `__repr__`.
19. Yes. Python objects are dynamic, so `obj.age = 30` simply adds a new entry to `__dict__`. It is blocked only in a few cases: when the class defines `__slots__`, when the attribute is a property without a setter, for frozen dataclasses, and for objects that have no `__dict__`, such as `object()` or built-in types.
20. `del obj` only removes the name `obj` and reduces the reference count; it does not directly delete the object. The object is freed when its reference count reaches 0 (CPython); objects stuck in reference cycles are cleaned up by the garbage collector. `__del__` is called just before the object is destroyed, but the timing is not guaranteed (and may not run at interpreter exit), so do not rely on it for cleanup; use context managers (`with`) instead.
21. Python implements encapsulation mostly by convention and trust, not by enforcement: (1) a single underscore (`_x`) marks an attribute as internal; (2) a double underscore (`__x`) triggers name mangling; (3) properties give controlled access; (4) data is kept inside the object and changed through methods. The idea is to hide implementation details and expose a clean public interface.
22. `_salary` is only a convention that means "internal, do not touch"; Python does not change anything. `__salary` is name-mangled inside the class to `_Employee__salary`. Neither is genuinely private, since both can still be reached from outside; the mangled one just makes accidental access harder.
23. Name mangling means that inside a class, any identifier like `__name` (two leading underscores, at most one trailing) is rewritten by the interpreter to `_ClassName__name`. It is used to avoid accidental name clashes between a parent class and its subclasses. It is not a security feature.
24. Yes, through the mangled name. It prints 50000. This is still encapsulation because encapsulation means bundling data with the methods that work on it and hiding internal details behind a public interface. The underscore and mangling clearly signal "do not use this", and anyone who bypasses it knowingly takes the risk of breaking the class.

```
e = Employee()
print(e._Employee__salary)
```

25. No. It is a different philosophy, not a weaker one. Python trusts developers ("we are all consenting adults") and relies on convention, while languages like Java enforce access at compile time (and even there reflection can bypass `private`). Real encapsulation is about design: a clear public interface and hidden internals that can change without breaking users. Python provides the tools (underscores, mangling, properties) and leaves the discipline to the programmer.

26. `@property` lets a method be accessed like an attribute. The getter returns the value; the setter (defined with `@balance.setter`) runs whenever you assign, so it can validate the value. Here `acc.balance = 100` works, but `acc.balance = -5` raises `ValueError`. The real data is stored in the internal attribute `_balance`.
27. With a plain public attribute, adding validation later would force you to change how callers use it. A property keeps the same syntax (`obj.attr`) while letting you add validation, computed values or logging later, so existing code does not break. So you can start simple with a public attribute and convert it to a property only when needed.
28. `__slots__` declares a fixed set of allowed attributes. Python then does not create a `__dict__` for each instance and stores the attributes in a compact fixed layout instead. Result: less memory and slightly faster attribute access, but you cannot add attributes that are not listed (you get `AttributeError`), and there is no `obj.__dict__` unless you include it in the slots.
29. It prints `{"_Parent__x": 1, "_Child__x": 2}`. The `__x` written inside `Parent` becomes `_Parent__x`, and the one inside `Child` becomes `_Child__x`, so they are two different attributes and neither overwrites the other. This shows that name mangling protects a class's attributes from accidental clashes in subclasses.
30. Keep the balance in an internal attribute (`_balance` or `__balance`) and expose it only through a read-only property (a getter with no setter). All changes go through methods like `deposit(amount)` and `withdraw(amount)` that validate the input (amount must be positive, no overdraft) and raise exceptions for invalid operations. Helper logic and the transaction history are kept internal with underscores. Because every change passes through one place with rules, the object always stays in a valid state, which is the real purpose of encapsulation.